Vektor Technical Report: RAG and Vector Search Architecture

Section 1: Introduction to Vector Search in Modern RAG Pipelines

Retrieval-Augmented Generation (RAG) combines dense semantic retrieval with large language models to provide accurate, grounded answers. Unlike keyphrase searching, dense vector retrieval converts unstructured text chunks into high-dimensional numerical embeddings using deep neural networks. The primary challenge in production RAG systems is balancing high query throughput, low search latency, and memory efficiency across large-scale document collections.

Section 2: Hierarchical Navigable Small World (HNSW) Graph Indexing

Vektor utilizes a Hierarchical Navigable Small World (HNSW) graph for approximate nearest neighbor (ANN) retrieval. HNSW constructs a multi-layer graph where upper layers contain long-range skip edges for logarithmic greedy routing, while the bottom layer contains a dense proximity graph. Key hyper-parameters governing HNSW performance include M (max number of bi-directional connections per node) and efConstruction (size of dynamic candidate list during build). Increasing M improves recall@10 on complex metric spaces but increases index memory footprint.

Section 3: Memory Layout and SIMD Vector Distance Computations

To achieve maximum hardware utilization during vector distance calculations, Vektor arranges vector storage in contiguous 64-byte cache-aligned memory blocks. Distance metrics such as Euclidean distance (L2), Cosine similarity, and Inner Product (IP) are accelerated using AVX-512 and FMA (Fused Multiply-Add) vector intrinsics. By enforcing contiguous memory layout and avoiding pointer indirection during distance loops, Vektor achieves up to 4x throughput improvements over naive pointer-array implementations.

Section 4: Vector Quantization and Product Quantization (PQ)

When collection sizes exceed system RAM, Vektor employs Scalar Quantization (SQ8) and Product Quantization (PQ) to compress high-dimensional floating-point vectors into compact byte codes. Product Quantization divides a high-dimensional vector space into lower-dimensional sub-spaces, quantizing each sub-space independently into cluster centroids. This compression reduces RAM requirements by up to 80% while retaining over 95% of top-k retrieval accuracy for similarity search operations.

Section 5: Empirical Evaluation and Semantic Relevance Methodology

Evaluating a vector database requires separating mathematical recall from semantic relevance. Mathematical recall@k verifies that the ANN index returns the same nearest neighbors as brute-force

exact search. In contrast, semantic relevance measures whether the retrieved text chunks satisfy human user queries. To evaluate semantic accuracy rigorously, researchers use human-labeled golden query sets scored with Precision@k metrics, independent of model self-evaluation.

Section 6: Conclusion and System Recommendations

Vektor demonstrates that combining optimized C++/Rust core index execution with clean Python interfaces provides both developer agility and high retrieval performance. For production RAG workloads, configuring HNSW with M=16 and ef_search=64 offers an optimal tradeoff between sub-millisecond query latency and high precision retrieval.